

프로그래밍은 완전히 자동화될 수 있는가

김현승, KAIST 전산학부

이 글은 프로그래밍을 완전히 자동화할 수 있는지를 계산불가능성(undecidability)과 프로그램의 올바름(correctness)이라는 관점에서 살펴본다. 모든 명세(specification)를 만족하는 프로그램을 자동으로 만들어 주는 프로그램 생성기는 계산불가능한 문제 때문에 존재할 수 없으며, 주어진 명세가 구현 가능한지를 항상 정확하게 판단하는 것 역시 불가능하다. 또한 프로그램의 올바름은 상황과 목적에 따라 달라질 수 있고, 어떤 기준은 수학적으로 엄밀하게 정의하기 어렵다. 따라서 프로그래밍 자동화의 목표는 인간을 완전히 대신하는 것이 아니라, 자동화할 수 있는 범위와 보장할 성질을 분명히 정하고 그 안에서 신뢰할 수 있는 결과를 제공하는 데 있다.

인간이 프로그래밍을 하는 이유의 핵심에는 계산과 절차의 자동화가 있다. 그렇다면 프로그래밍이라는 절차 자체도 완전히 자동화할 수 있을까? 최근 생성형 인공지능이 급격히 발전하면서 자동화가 머지않아 가능할 것처럼 보이기도 한다. 그러나 프로그래밍의 완전한 자동화는 근본적으로 불가능하다. 연산 능력이 부족하거나 알고리즘이 아직 충분히 발전하지 않았기 때문이 아니다. 모든 문제를 프로그램으로 해결할 수 없다는 계산불가능성의 한계가 존재하기 때문이다. 무엇이 ‘올바른 프로그램’인지 결정하는 일 자체도 완전한 자동화를 어렵게 한다.

계산불가능성은 모든 명세를 받아 올바른 프로그램으로 바꾸어 주는 자동 프로그램 생성기가 존재할 수 없음을 보여준다. 임의의 명세 S 를 입력받아 이를 만족하는 프로그램 P 를 출력하는 자동 프로그램 생성기가 존재한다고 가정해 보자. 그러나 명세가 임의의 프로그램이 주어진 입력에서 정지하는지를 항상 판정하는 문제와 같은 계산불가능한 문제의 해결을 요구한다면, 그러한 명세를 만족하는 프로그램은 존재할 수 없다. 따라서 아무리 뛰어난 자동 프로그램 생성기라도 모든 명세를 프로그램으로 바꿀 수는 없다.

구현 가능한 명세만을 골라 자동으로 프로그램을 만드는 방법 역시 완전한 자동화의 해결책이 될 수 없다. 이를 위해서는 먼저 주어진 명세가 구현 가능한지를 판단하고, 만들어진 프로그램이 실제로 그 명세를 만족하는지도 판단해야 한다. 그러나 임의의 프로그램과 명세를 대상으로 이러한 판단을 항상 정확하게 할 수 있다면, 정지 문제(halting problem) 역시 해결할 수 있게 된다. 이는 정지 문제가 계산불가능하다는 사실과 모순된다. 물론 명세와 프로그램의 범위를 제한하면 이러한 판단을 계산가능하게 만들 수 있다. 하지만 이때는 자동화할 수 있는 프로그램의 범위 역시 함께 제한된다.

계산불가능성의 문제를 피하더라도 ‘어떤 프로그램이 올바른 프로그램인가’라는 더 근본적인 문제가 남는다. 프로그램의 올바름은 상황과 목적에 따라 서로 다른 기준으로 판단될 수 있다. 어떤 상황에서는 계산 결과의 정확성이 무엇보다 중요하지만, 다른 상황에서는 약간의 오차를 허용하더라도 정해진 시간 안에 결과를 내놓는 것이 더 중요할 수 있다. 어떤 사용자는 높은 보안성을 중시하는 반면, 다른 사용자는 어느 정도의 위험을 감수하더라도 편의성을 더 중요하게 생각할 수 있다. 즉 하나의 프로그램을 두고도 무엇을 ‘올바르다’고 볼 것인지가 달라질 수 있다.

프로그래밍을 완전히 자동화하기 어려운 또 다른 이유는 프로그램이 만족해야 할 조건 자체를 항상 명확하게 정할 수 있는 것이 아니기 때문이다. 프로그램의 올바름에는 수학적으로 엄밀하게 정의하기 어려운 기준이 포함되기도 한다. 번역 프로그램을 생각해 보자. 좋은 번역 프로그램은 단순히 원문의 뜻을 전달하는 것을 넘어, 번역된 글을 읽는 사람이 자연스럽게 받아들일 수 있는 문장을 만들어야 한다. 그러나 ‘자연스럽게 받아들일 수 있는 문장’이 무엇인지 모든 경우에 적용할 수 있도록 엄밀하게 정의하는 것은 쉽지 않다. 이처럼 프로그래밍에서 어려운 일은 코드를 작성하는 행위만이 아니라, 무엇을 만들어야 하는지 결정하고 그 프로그램이 만족해야 할 조건을 정하는 일에도 있다.

완전한 자동화가 불가능한 상황에서는 자동화의 범위를 넓히는 것보다, 자동화된 시스템이 제한된 범위 안에서 신뢰할 수 있는 결과를 내도록 하는 것이 더 중요하다. 모든 명세를 다루는 프로그램 생성기는 존재할 수 없지만, 다루는 범위를 제한하면 구현 가능성과 올바름을 판단할

수 있는 경우가 생긴다. 이때 프로그램의 모든 성질을 확인하려 하기보다, 타입이나 메모리의 안전성과 같이 반드시 보장해야 할 성질을 명확히 정할 수도 있다. 이처럼 자동화의 대상과 보장할 성질을 분명히 정하면, 다룰 수 있는 문제의 범위는 좁아지더라도 그 결과에 대해서는 더 확실한 보장을 할 수 있다.

계산불가능한 성질을 다뤄야 하는 경우에는 올바른 프로그램을 모두 찾아내는 것보다, 판정 결과를 신뢰할 수 있도록 하는 것이 더 중요하다. 모든 프로그램을 대상으로 어떤 성질을 판정하려 하면 안전성(soundness)과 완전성(completeness)을 동시에 만족하는 방법을 만들 수 없는 경우가 있다. 이때 판정 결과 자체를 믿을 수 없다면 자동화된 결과를 사람이 다시 확인해야 하므로, 자동화의 의미도 크게 줄어든다. 따라서 실제로 올바른 프로그램을 일부 놓치더라도, 올바르다고 판정한 프로그램만큼은 실제로 올바르도록 안전성을 우선하는 편이 더 적절하다.

프로그래밍이라는 절차 전체를 완전히 자동화하는 것은 불가능하다. 계산불가능한 문제가 존재하고, 프로그램이 무엇을 만족해야 하는지 정하는 일에도 인간의 판단이 필요하기 때문이다. 그러나 이것이 프로그래밍 자동화의 실패를 의미하지는 않는다. 자동화의 범위를 분명히 정하고, 그 안에서 신뢰할 수 있는 결과를 보장하는 방향으로 발전시킬 수 있다. 프로그래밍 자동화의 목표는 인간을 완전히 대신하는 것이 아니라, 사람이 판단해야 할 부분과 기계에 맡길 수 있는 부분을 구분하고 후자를 믿고 맡길 수 있도록 하는 데 있다.